[Lecture Note] Evaluating Predictive Performance

MGS3701 Data Mining, Spring 2025

Chad (Chungil Chae)

Sun, 25 May 2025

Learning Objectives

1. **Importance of Performance Assessment:**
    - Avoid overfitting to training data.
    - Test model performance on data not used in the training step.
2. **Performance Metrics:**
    - **Prediction:**
        - Average Error
        - Mean Absolute Percentage Error (MAPE)
        - Root Mean Squared Error (RMSE)
    - **Classification:**
        - Metrics based on the confusion matrix:
            - Overall accuracy
            - Specificity
            - Sensitivity
            - Metrics accounting for misclassification costs
3. **Cutoff Value and Classification Performance:**
    - Choice of cutoff value impacts classification performance.
    - **ROC Curve:**
        - Popular chart for assessing method performance at different cutoff values.
4. **Ranking vs. Classification:**
    - **Ranking:**
        - Goal: Accurately classify the most interesting or important records.
        - **Lift Charts:**
            - Used to assess performance in ranking tasks.
5. **Handling Rare Classes:**

- Need for oversampling rare classes.
- Adjust performance metrics for the oversampling.

6. **Detecting Overfitting:**
   - Compare metrics based on validation data to those based on training data.
   - Extreme differences between these metrics can indicate overfitting.

SHORT VIDEO INTRODUCTION

https://www.youtube.com/watch?v=LbX4X71-TFI

Introduction

---

1. **Outcome Types:**
   - **Predicted Numerical Value:**
     - Outcome variable is numerical (e.g., house price).
   - **Predicted Class Membership:**
     - Outcome variable is categorical (e.g., buyer/nonbuyer).
   - **Propensity:**
     - Probability of class membership for a categorical outcome (e.g., propensity to default).
2. **Methods:**
   - **Prediction Methods:**
     - Used for generating numerical predictions.
   - **Classification Methods ("Classifiers"):**
     - Used for generating propensities.
     - By applying a cutoff value on propensities, generate predicted class memberships.
3. **Predictive Uses of Classifiers:**
   - **Classification:**
     - Aimed at predicting class membership for new records.
   - **Ranking:**
     - Detecting among a set of new records the ones most likely to belong to a class of interest.
4. **Evaluating Prediction Methods:**
   - **Section 5.2:**
     - Approach for judging the usefulness of a prediction method used for generating numerical predictions.

- **Section 5.3:**
  - Approach for judging the usefulness of a classifier used for classification.
- **Section 5.4:**
  - Approach for judging the usefulness of a classifier used for ranking.
- **Section 5.5:**
  - Evaluating performance under the scenario of oversampling.

Evaluating Predictive Performance

Predictive Accuracy vs. Goodness-of-Fit

1. **Distinction:**
   - **Predictive Accuracy:**
     - Assesses how well the model predicts outcome for new records.
   - **Goodness-of-Fit:**
     - Assesses how well the model fits the training data.
2. **Classical Statistical Measures:**
   - Measures such as R^2 and standard error of estimate are used in classical regression modeling.
   - Residual analysis gauges goodness-of-fit.
   - These measures do not indicate the model's ability to predict new records.
3. **Assessing Prediction Performance:**
   - **Use of Validation Set:**
     - Validation set serves as an objective ground to assess predictive accuracy.
     - More similar to future records to be predicted.
     - Not used to select predictors or to estimate model parameters.
   - **Process:**
     - Models are trained on the training data.
     - Applied to the validation data.
     - Accuracy measures are based on prediction errors from the validation set.

Naive Benchmark: The Average

Benchmark Criterion in Prediction

1. **Naive Benchmark:**

- The prediction for a new record is the average outcome value of the records in the training set ($\overline{y}$).
- This method ignores all predictor information.

2. **Purpose:**
   - Serves as a baseline to measure the performance of predictive models.
   - Sometimes referred to as a naive benchmark.
3. **Expectation:**
   - A good predictive model should outperform the naive benchmark in terms of predictive accuracy.

Prediction Accuracy Measure

1. **Prediction Error for Record** i:
   - Defined as the difference between the actual outcome value and the predicted outcome value: $e_i = y_i - \hat{y}_i$
2. **Popular Accuracy Measures:**
   - **Mean Absolute Error (MAE):** $\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|e_i|$
     - Magnitude of the average absolute error.
   - **Mean Error:** $\text{Mean Error} = \frac{1}{n}\sum_{i=1}^{n} e_i$
     - Similar to MAE but retains the sign of the errors.
     - Indicates whether predictions are on average over- or underpredicting the outcome variable.
   - **Mean Percentage Error (MPE):** $\text{MPE} = 100 \times \frac{1}{n}\sum_{i=1}^{n}\frac{e_i}{y_i}$
     - Percentage score of how predictions deviate from actual values, considering the direction of the error.
   - **Mean Absolute Percentage Error (MAPE):** $\text{MAPE} = 100 \times \frac{1}{n}\sum_{i=1}^{n}\left|\frac{e_i}{y_i}\right|$
     - Percentage score of how predictions deviate from actual values on average.
   - **Root Mean Squared Error (RMSE):** $\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} e_i^2}$
     - Similar to the standard error of estimate in linear regression.
     - Computed on validation data.
     - Has the same units as the outcome variable.
3. **Using Accuracy Measures:**
   - Compare models.
   - Assess prediction accuracy.
   - Note: All measures are influenced by outliers.
4. **Handling Outliers:**
   - Compute median-based measures for comparison.

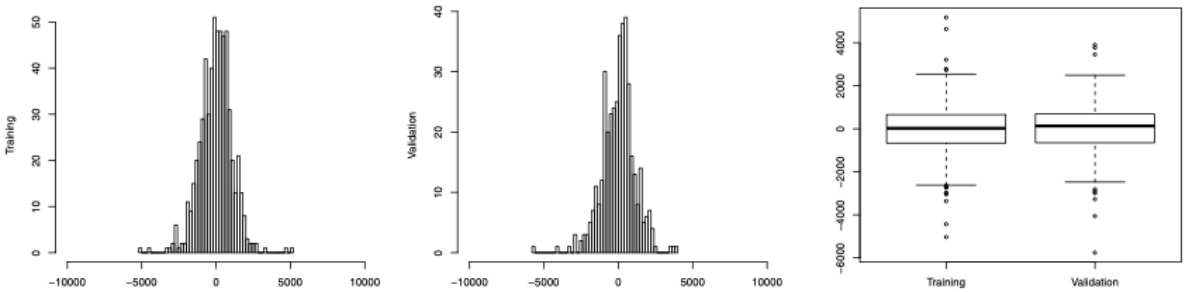
- Plot histogram or boxplot of the errors to check outlier influence.

5. **Visualizing Prediction Errors:**
   - Plotting the distribution of prediction errors is useful.
   - Histogram and boxplots can reveal more information than metrics alone.
6. **Illustration with Example Data:**
   - **Example: Prices of Used Toyota Corolla Cars:**
     - Training set: 600 cars.
     - Validation set: 400 cars.
     - **Results:**
       * Error metrics and charts show errors are mostly in the [−1000, 1000] range.
       * A few large positive (under-prediction) errors are present.
     - **Table 5.1 and Figure 5.1:** Display results for training and validation sets.

```
# package forecast is required to evaluate performance
library(forecast)

# load file
toyota.corolla.df <- read.csv("ToyotaCorolla.csv")

# randomly generate training and validation sets
training <- sample(toyota.corolla.df$Id, 600)
validation <- sample(setdiff(toyota.corolla.df$Id, training), 400)

# run linear regression model
reg <- lm(Price~., data=toyota.corolla.df[,-c(1,2,8,11)], subset=training,
          na.action=na.exclude)
pred_t <- predict(reg, na.action=na.pass)
pred_v <- predict(reg, newdata=toyota.corolla.df[validation,-c(1,2,8,11)],
                  na.action=na.pass)

## evaluate performance
# training
accuracy(pred_t, toyota.corolla.df[training,]$Price)
# validation
accuracy(pred_v, toyota.corolla.df[validation,]$Price)
```

Output

```
> # training
> accuracy(pred_t, toyota.corolla.df[training,]$Price)
                      ME     RMSE      MAE        MPE     MAPE
Test set -3.542119e-11 1012.578 784.1166 -0.8180893 7.773598

> # validation
> accuracy(pred_v, toyota.corolla.df[validation,]$Price)
                 ME     RMSE      MAE       MPE     MAPE
Test set -107.8089 1262.623 833.3621 -2.345028 8.846669
```

Comparing Traning and Validation Performance

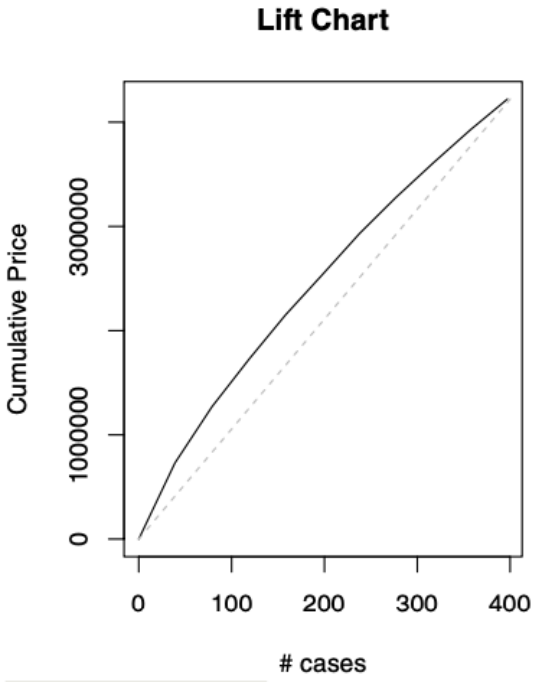
1. **Training Errors vs. Prediction Errors:**
   - **Training Errors:**
     - Based on the training set.
     - Indicate model fit to the training data.
   - **Prediction Errors:**
     - Based on the validation set.
     - Measure the model's ability to predict new data (predictive performance).
2. **Expected Behavior:**
   - Training errors are generally smaller than validation errors because the model was fitted using the training set.
   - More complex models are more likely to overfit the training data.
3. **Overfitting:**
   - Indicated by a greater difference between training and validation errors.
   - Extreme overfitting: Training errors are zero (perfect fit), while validation errors are non-zero and non-negligible.
4. **Importance of Comparison:**
   - Compare error plots and metrics (e.g., RMSE, MAE) of the training and validation sets.
   - This helps identify overfitting and assess model performance.
5. **Illustration with Example:**
   - **Table 5.1:** Shows comparison of performance measures for the training and validation sets.
   - **Figure 5.1:** Charts provide more insight:
     - Discrepancies between training and validation errors are likely due to outliers in the validation set.
     - Positive validation errors (under-predictions) are slightly larger, reflected by medians and outliers.

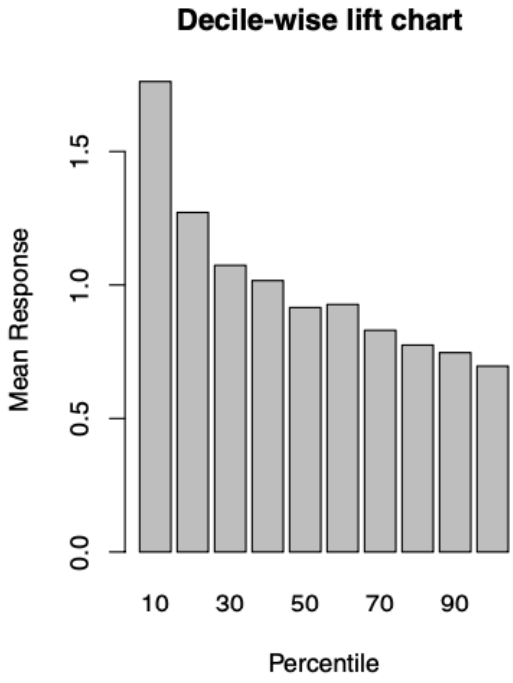
Lift Chart

Lift Charts for Assessing Predictive Performance

1. **Goal:**
    - Search for a subset of records that gives the highest cumulative predicted values.
    - **Ranking:** Identifying records with the highest predicted values.
2. **Lift Chart:**
    - Compares model's predictive performance to a baseline model with no predictors.
    - Relevant when searching for a set of records with the highest cumulative predicted values.
    - Not relevant for predicting individual outcome values.
3. **Example Scenario:**
    - **Car Rental Firm:**
        - Disposes of used vehicles regularly.
        - Selects cars for resale through its own channels to maximize revenue.
        - Lift chart used to predict revenue lift for selected cars.
4. **Constructing a Lift Chart:**
    - Order records by predicted value from high to low (typically validation data).
    - Accumulate actual values and plot cumulative values on the y-axis as a function of the number of records accumulated (x-axis).
    - Compare to a naive prediction with the average outcome value ($\overline{y}$) for each record, resulting in a diagonal line.
    - The further the lift curve is from the diagonal line, the better the model separates high-value outcomes from low-value outcomes.
5. **Decile Lift Chart:**
    - Group ordered records into ten deciles.
    - Chart presents the ratio of model lift to naive benchmark lift for each decile.
6. **Illustration with Example Data:**
    - **Figure 5.2:** Shows lift chart and decile lift chart for a linear regression model fitted on Toyota data (validation set of 400 cars).
    - **Observation:**
        - Model's lift curve is higher than baseline, indicating better performance.
        - **Scenario:** Choosing the top 10% of cars with highest predicted sales provides 1.7 times the revenue compared to randomly selecting 10% of cars.
        - **Calculation:**
            * Sales for 40 random cars: \$486,871 (from baseline curve at x = 40).
            * Actual sales for 40 cars with highest predicted values: \$835,883 (from lift curve at x = 40).

- Ratio: $\frac{835,883}{486,871} = 1.7$.





Judging Classifier Performance

Importance of Performance Measures

1. **Variety of Classifiers and Predictive Methods:**
   - Wide range of classifiers and predictive methods available.
   - Different methods and options within a single method can yield different results.
2. **Need for Performance Measures:**
   - Essential to measure success before choosing algorithms and setting options.
3. **Criteria for Classifier Performance:**
   - **Misclassification Error:**
     - A natural criterion for judging performance.
     - Misclassification occurs when a record is classified into the wrong class.
     - Goal is to minimize the probability of misclassification.
4. **Real-World Classifiers:**

- Perfect classifiers (no misclassification errors) are unrealistic.
- **Reasons:**
  - Presence of "noise" in the data.
  - Lack of complete information needed for precise classification.

5. **Minimal Probability of Misclassification:**

- There is no universally accepted minimal probability of misclassification.
- The acceptable level depends on the specific application and context of the classification task.

Benchmark: The Naive Rule

1. **Definition:**

- A simple rule for classifying a record into one of m classes by:
  - Ignoring all predictor information ($x_1, x_2, \ldots, x_p$).
  - Classifying the record as a member of the majority class (the most prevalent class).

2. **Purpose:**

- Used as a baseline or benchmark for evaluating the performance of more complicated classifiers.

3. **Expectation:**

- A classifier that uses additional predictor information should outperform the naive rule.

4. **Performance Measures:**

- Performance measures based on the naive rule assess how much better a classifier performs compared to the naive rule.
- **Example: Multiple R^2**:
  - Measures the distance between the fit of the classifier to the data and the fit of the naive rule to the data.
  - Further details can be found in Section 10.6.

5. **Comparison to Numerical Outcomes:**

- Similar to using the sample mean ($\overline{y}$) as a naive benchmark for numerical outcomes.
- The naive rule for classification relies solely on y information and excludes additional predictor information.

Class Separation

Impact of Predictor Information on Classification

1. **Class Separation by Predictors:**

- **Well Separated Classes:**

  - Even a small dataset can help in finding a good classifier.
- **Poorly Separated Classes:**
  - Even a very large dataset will not help if classes are not well separated by the predictors.

2. **Illustration (Figure 5.3):**

- **Two-Class Case:**
  - **Top Panel:**
    * Small dataset ($n = 24$ records).
    * Predictors: Income and lot size.
    * Goal: Separate owners from nonowners.
    * **Observation:** Predictor information seems useful, separating the two classes (owners/nonowners) effectively.
  - **Bottom Panel:**
    * Large dataset ($n = 5000$ records).
    * Predictors: Income and monthly average credit card spending.
    * Goal: Separate loan acceptors from nonacceptors.
    * **Observation:** Predictors do not separate the two classes well in most of the higher ranges (loan acceptors/nonacceptors).

3. **Conclusion:**

- The effectiveness of predictors in separating classes is crucial for good classification performance.
- The size of the dataset alone is not sufficient to ensure good classification if the predictors do not provide good separation.

The Confusion Matrix

1. **Definition:**

- Also called a classification matrix.
- Summarizes correct and incorrect classifications made by a classifier for a certain dataset.

2. **Structure:**

- **Rows:** Predicted classes.
- **Columns:** True (actual) classes.

3. **Example (Table 5.2):**

- Two-class (0/1) problem applied to 3000 records.
- **Diagonal Cells:**
  - Upper left and lower right: Correct classifications (predicted class matches actual class).
- **Off-Diagonal Cells:**
  - Lower left: Number of class 1 members misclassified as 0 (85 misclassifications in the example).

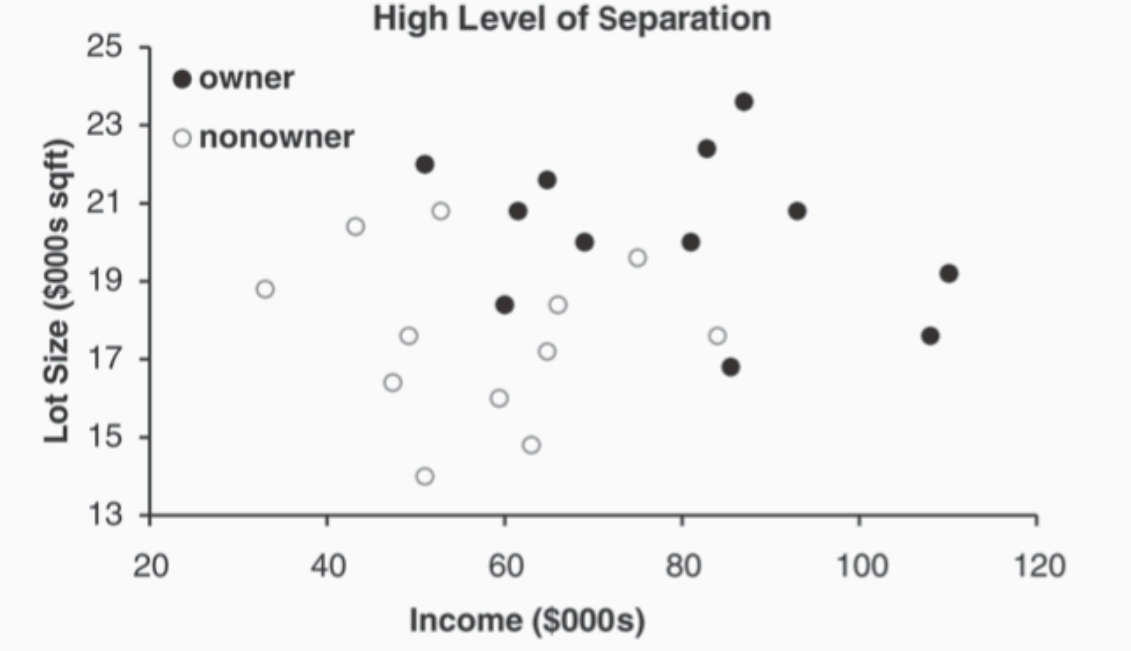
  - Upper right: Number of class 0 members misclassified as 1 (25 misclassifications in the example).

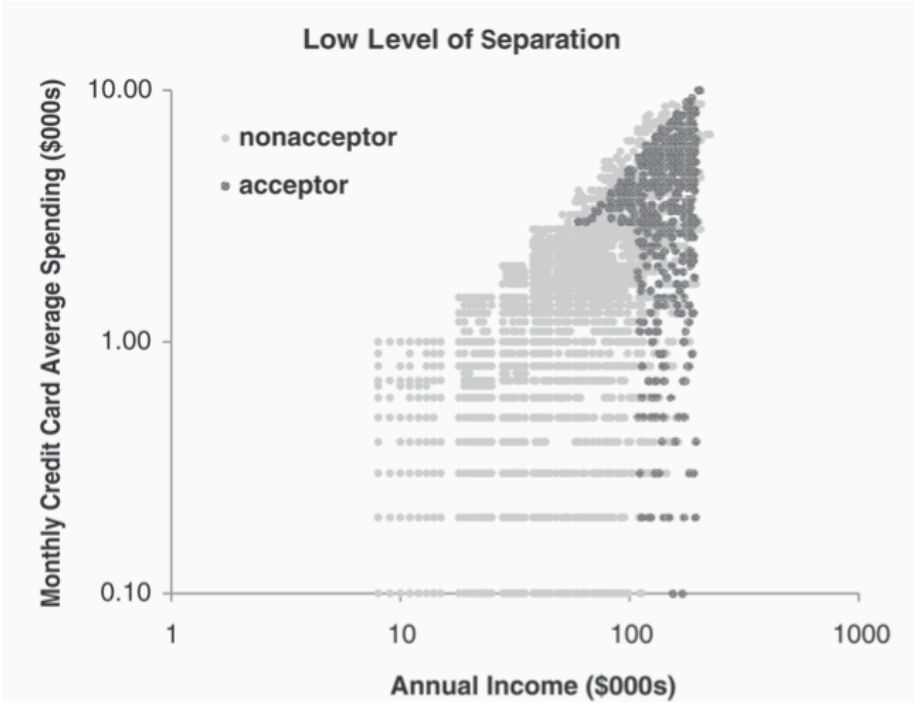
4. **Creating a Confusion Matrix in R:**

   - Use the `confusionMatrix()` function in the `caret` package.
   - This function creates a cross-tabulation of actual and predicted classes.
   - **Example:**
     - An example will be shown later in the chapter.

5. **Estimates from the Confusion Matrix:**

   - Provides estimates of true classification and misclassification rates.
   - Reliable estimates if the dataset is large and neither class is very rare.
   - Public data (e.g., US Census data) can sometimes be used for estimating proportions.
   - In most business settings, exact class proportions may not be known.





297

		Actual Class	
		0	1
Predicted Class	0	2689	85
	1	25	201

298

299 **Using the Validation Data**

300 **Honest Estimation of Future Classification Error**

301 1. **Using Validation Data:**

302 • Obtain an honest estimate of future classification error by using the confusion matrix computed
303 from the validation data.

- **Process:**
  - Partition the data into training and validation sets by random selection of records.
  - Construct a classifier using the training data.
  - Apply the classifier to the validation data to yield predicted classifications for validation records.
  - Summarize the results in a confusion matrix.

2. **Importance of Validation Data:**

- Confusion matrix from the training data is not useful for estimating the misclassification rate for new data due to overfitting risk.
- Validation data provide an objective measure of classification performance on new data.

3. **Comparison for Overfitting Detection:**

- Compare the confusion matrix from the training data to the confusion matrix from the validation data.
- Expected: Somewhat inferior results on validation data compared to training data.
- Large discrepancies in performance between training and validation data may indicate overfitting.

4. **Steps Summarized:**

- **Step 1:** Randomly partition data into training and validation sets.
- **Step 2:** Train the classifier using the training data.
- **Step 3:** Apply the classifier to the validation data.
- **Step 4:** Summarize predicted classifications in the validation data confusion matrix.
- **Step 5:** Compare training data confusion matrix with validation data confusion matrix to assess overfitting.

Accuracy Measures

Accuracy Measures from the Confusion Matrix

1. **Confusion Matrix Notation:**

- Consider a two-class case with classes C_1 and C_2 (e.g., buyer/non-buyer).
- **Schematic Confusion Matrix (Table 5.3):**
  - $n_{i,j}$: Number of records that are class C_i members and classified as C_j members.
  - If $i \neq j$, these are counts of misclassifications.
  - Total number of records: $n = n_{1,1} + n_{1,2} + n_{2,1} + n_{2,2}$.

2. **Misclassification Rate (Overall Error Rate):**

- Measures the proportion of misclassified records.
- **Formula:**

$$\text{err} = \frac{n_{1,2} + n_{2,1}}{n}$$

- **Example Calculation (Table 5.2):**

$$\text{err} = \frac{25 + 85}{3000} = 3.67\%$$

3. **Overall Accuracy:**

- Measures the proportion of correctly classified records.
- **Formula:**

$$\text{accuracy} = 1 - \text{err} = \frac{n_{1,1} + n_{2,2}}{n}$$

- **Example Calculation (Table 5.2):**

$$\text{accuracy} = \frac{201 + 2689}{3000} = 96.33\%$$

		Actual Class	
		C_1	**C_2**
Predicted Class	**C_1**	$n_{1,1}$ = number of C_1 records classified correctly	$n_{2,1}$ = number of C_2 records classified incorrectly as C_1
	C_2	$n_{1,2}$ = number of C_1 records classified incorrectly as C_2	$n_{2,2}$ = number of C_2 records classified correctly

Propensities and Cutoff for Classification

Estimating Class Probabilities and Cutoff Values

1. **Probabilities (Propensities):**

- Estimate the probability that a record belongs to each class.
- Used for:
  - Generating predicted class membership (classification).
  - Rank-ordering records by their probability of belonging to a class of interest.

2. **Assigning Classes Based on Probabilities:**

- Assign the record to the class with the highest probability if overall classification accuracy is of interest.
- Focus on a particular class and compare the propensity of belonging to that class against a cutoff value.

3. **Cutoff Value Approach:**

- **Two Classes (e.g., C1 and C2):**

- Default cutoff value: 0.5.
- Probability of being a class C_1:
  - If > 0.5, classify as C_1.
  - If < 0.5, classify as C_2.
- Adjusting cutoff:
  - Cutoff > 0.5: Fewer records classified as C_1.
  - Cutoff < 0.5: More records classified as C_1.

4. **Example with Different Cutoff Values:**

- **Data (Table 5.4):** Actual class for 24 records, sorted by the probability of being an "owner."
- **Standard Cutoff (0.5):**
  - Misclassification rate: $\frac{3}{24}$.
- **Lower Cutoff (0.25):**
  - Misclassification rate: $\frac{5}{24}$ (more nonowners misclassified as owners).
- **Higher Cutoff (0.75):**
  - Misclassification rate: $\frac{6}{24}$ (more owners misclassified as nonowners).
- **Classification Tables (Table 5.5):** Display results.

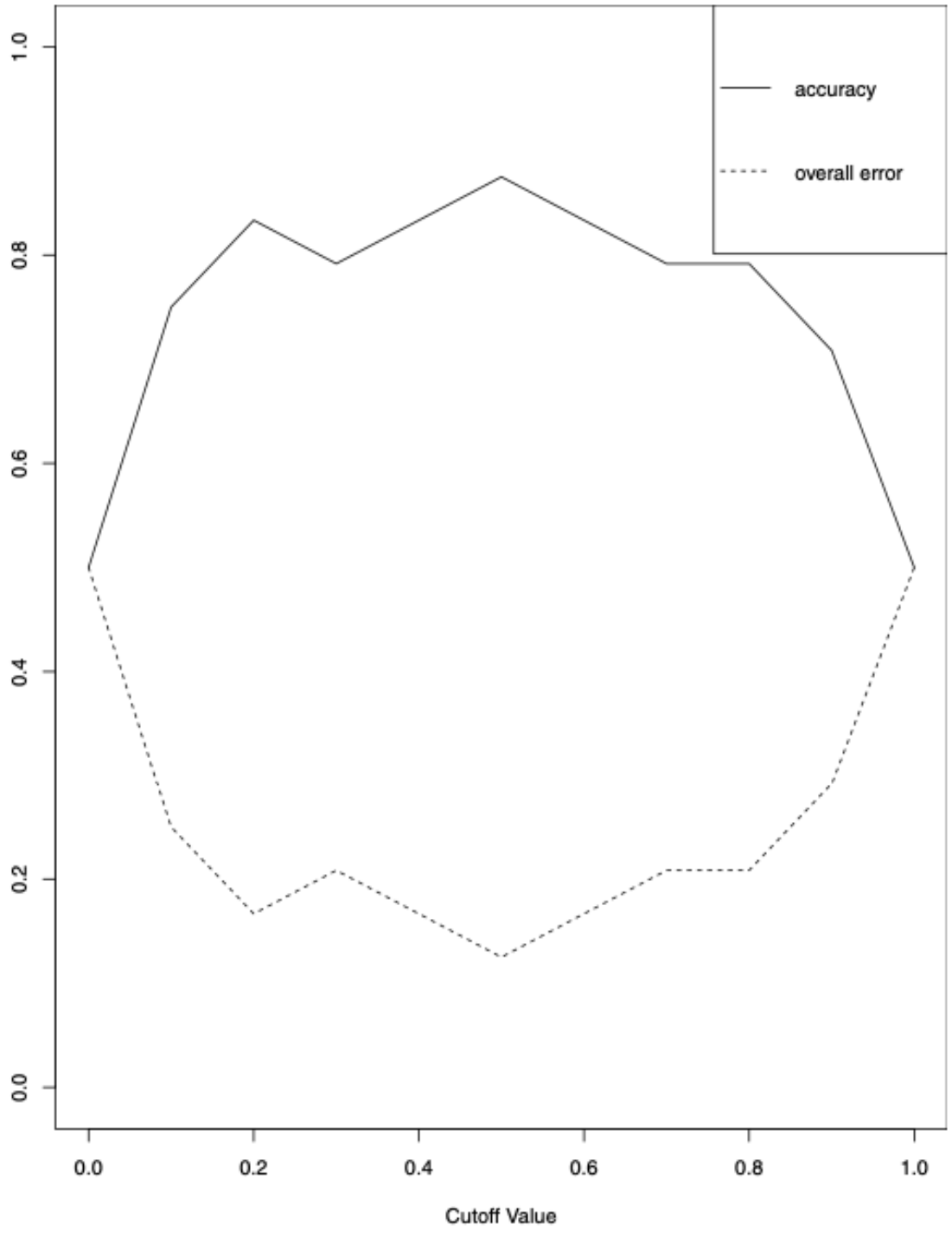
Actual Class	Probability of Class "owner"	Actual Class	Probability of Class "owner"
owner	0.9959	owner	0.5055
owner	0.9875	nonowner	0.4713
owner	0.9844	nonowner	0.3371
owner	0.9804	owner	0.2179
owner	0.9481	nonowner	0.1992
owner	0.8892	nonowner	0.1494
owner	0.8476	nonowner	0.0479
nonowner	0.7628	nonowner	0.0383
owner	0.7069	nonowner	0.0248
owner	0.6807	nonowner	0.0218
owner	0.6563	nonowner	0.0161
nonowner	0.6224	nonowner	0.0031

5. **Plotting Performance vs. Cutoff:**

- Shows how accuracy or misclassification rates change with cutoff value.
- **Example (Figure 5.4):**
  - Riding mowers data.
  - Accuracy level stable around 0.8 for cutoff values between 0.2 and 0.8.


```
> owner.df <- read.csv("ownerExample.csv")
## cutoff = 0.5
> confusionMatrix(ifelse(owner.df$Probability>0.5, 'owner', 'nonowner'), owner.df$Class)
# note: "reference" = "actual"
Confusion Matrix and Statistics

          Reference
Prediction nonowner owner
  nonowner       10     1
  owner           2    11

               Accuracy : 0.875

## cutoff = 0.25
> confusionMatrix(ifelse(owner.df$Probability>0.25, 'owner', 'nonowner'), owner.df$Class)
Confusion Matrix and Statistics

          Reference
Prediction nonowner owner
  nonowner        8     1
  owner           4    11

               Accuracy : 0.7916667

## cutoff = 0.75
> confusionMatrix(ifelse(owner.df$Probability>0.75, 'owner', 'nonowner'), owner.df$Class)
Confusion Matrix and Statistics

          Reference
Prediction nonowner owner
  nonowner       11     5
  owner           1     7

               Accuracy : 0.75
```

```
# create empty accuracy table
accT = c()

# compute accuracy per cutoff
for (cut in seq(0,1,0.1)){
  cm <- confusionMatrix(1 * (df$prob > cut), df$actual)
  accT = c(accT, cm$overall[1])
}

# plot accuracy
plot(accT ~ seq(0,1,0.1), xlab = "Cutoff Value", ylab = "", type = "l", ylim = c(0, 1))
lines(1-accT ~ seq(0,1,0.1), type = "l", lty = 2)
legend("topright",  c("accuracy", "overall error"), lty = c(1, 2), merge = TRUE)
```

6. **Reasons for Adjusting Cutoff Values:**

   - Importance of classifying one class (e.g., owners) properly over another.
   - Costs of misclassification might be asymmetric:

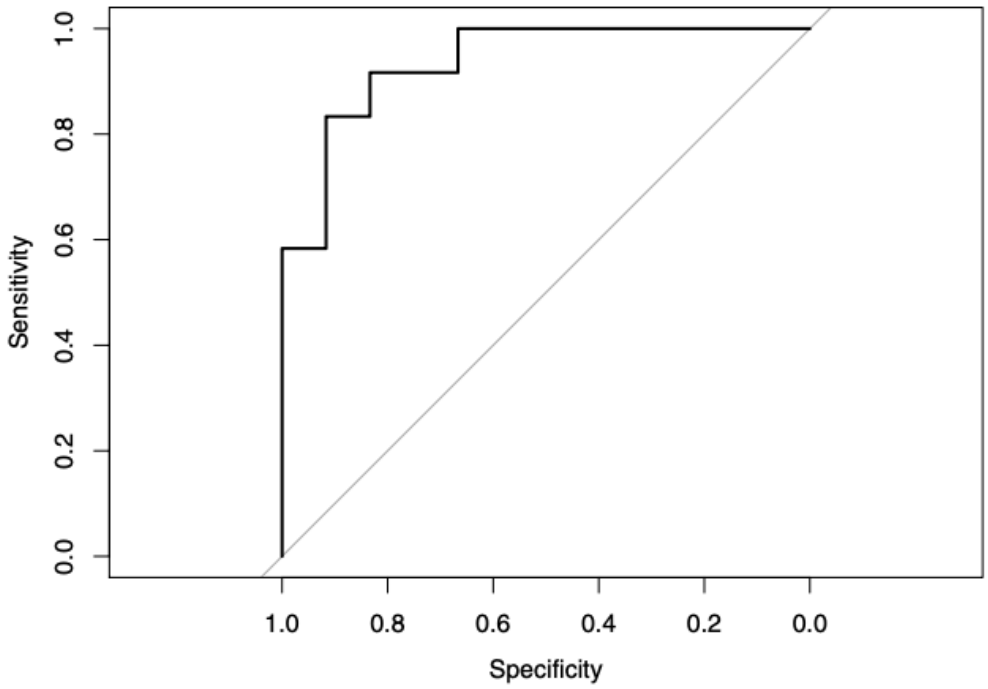
  - Adjust cutoff to classify more records as the high-value class.
  - Accept more misclassifications where the cost is low.
- Adjustment done after selecting the data mining model.
- Possible to incorporate costs before deriving the model (discussed in detail further on).

Performance in Case of Unequal Importance of Classes

Sensitivity, Specificity, and ROC Curve

1. **Importance of Predicting Class Membership Correctly:**
   - Sometimes more critical for one class (e.g., predicting bankruptcy accurately).
2. **Sensitivity and Specificity:**
   - **Sensitivity (Recall):**
     - Ability to detect the important class members correctly.
     - Formula: $\frac{n_{1,1}}{n_{1,1}+n_{1,2}}$
     - Percentage of $C_1 $ members classified correctly.
   - **Specificity:**
     - Ability to rule out $C_2 $ members correctly.
     - Formula: $\frac{n_{2,2}}{n_{2,1}+n_{2,2}}$
     - Percentage of $C_2 $ members classified correctly.
3. **Using Sensitivity and Specificity:**
   - Plot sensitivity and specificity against the cutoff value to find a balance.
4. **ROC Curve:**
   - **Receiver Operating Characteristic Curve:**
     - Plots {sensitivity, specificity} pairs as the cutoff value descends from 1 to 0.
     - **Alternative Presentation:**
       * Plot 1-specificity on the x-axis, sensitivity on the y-axis.
       * 0 on the left end of the x-axis, 1 on the right.
     - **Interpretation:**
       * Better performance is reflected by curves closer to the top-left corner.
       * Comparison curve: Diagonal line reflects performance of the naive rule with varying cutoffs.
     - **Area Under the Curve (AUC):**
       * Summary metric of ROC curve.
       * Ranges from 1 (perfect discrimination) to 0.5 (no better than the naive rule).
5. **Example: Owner/Nonowner Classification:**
   - **Figure 5.5:** Shows the ROC curve and AUC for predicting owner/nonowner status.

- **AUC Interpretation:**
  - Higher AUC indicates better performance.



Asymmetric Misclassificaiton Costs

1. **Context:**
   - In some scenarios, misclassifying a record in one class (e.g., a potential buyer) incurs greater cost than misclassifying in another class (e.g., non-buyer).
   - Example: Predicting financial status (bankrupt/solvent).
2. **Misclassification Costs vs. Overall Accuracy:**
   - Misclassification rate as a criterion can be misleading if costs are asymmetric.
   - Assume a zero cost (or benefit) for making correct classifications to simplify assessment.
3. **Impact of Misclassification Costs:**
   - **Opportunity Cost Example:**
     - Misclassifying a potential buyer misses out on a sale.
     - Misclassifying a non-buyer incurs contact costs.
   - **Confusion Matrix Example:**
     - Random sampling with 1% response rate:
       - Classify everyone as non-responder (error rate = 1%).

    - A classifier that misclassifies 2% of buyers and 20% of non-buyers might have a higher error rate but be more profitable.

4. **Comparing Expected Costs:**
    - Compare classifiers based on overall expected costs (or profits).
5. **Profit and Cost Example:**
    - **Scenario:**
        - Offer mailed to 1000 people, 1% response rate.
        - Naively classifying everyone as non-responder:
            - Error rate = 1%, profit = $0.
        - Using a data mining routine:
            - Error rate = 2.2%(100 × (20 + 2)/1000 = 2.2%), profit = $60.
    - **Confusion Matrix:**
        - Predicted classifications:

	Actual 0	Actual 1
Predicted 0	970	2
Predicted 1	20	8

    - **Profit Matrix:**
        - Profit Calculation:

Profit	Actual 0	Actual 1
Predicted 0	0	0
Predicted 1	-20	80

    - **Cost Matrix:**
        - Cost Calculation:

Cost	Actual 0	Actual 1
Predicted 0	0	20
Predicted 1	20	8

6. **Average Misclassification Cost:**
    - **Formula:**

$$\text{Average Misclassification Cost} = \frac{q_1 \cdot n_{1,2} + q_2 \cdot n_{2,1}}{n}$$

    - **Minimization:**
        - Depends on the ratio of the costs q_2/q_1.
        - Practical since estimating the cost ratio is often easier than individual costs.

7. **Adjusting for Sample Proportions:**
    - If sample proportions differ from future/population proportions:
        - Incorporate true proportions $p(C1)$ and $p(C2)$.
    - **Adjusted Formula:**

$$\text{Average Misclassification Cost} = \frac{n_{1,2}}{p(C1)} \cdot q_1 + \frac{n_{2,1}}{p(C2)} \cdot q_2$$

    - **Optimization:**
        - Depends on ratios q_2/q_1 and $p(C2)/p(C1)$.

Generalization to More Than Two Classes

Extension to Multi-Class Classifiers

1. **General Concept:**
    - Comments and methodologies for two-class classifiers extend to classifiers with more than two classes.
    - Suppose we have m classes: $C_1, C_2, \ldots, C_m$.
2. **Confusion Matrix:**
    - For m classes, the confusion matrix becomes an $m \times m$ matrix.
    - **Diagonal Cells:**
        - Represent correct classifications.
        - Misclassification cost for diagonal cells is zero.
3. **Incorporating Prior Probabilities:**
    - Prior probabilities of the m classes are still incorporated in the same manner as for two-class classifiers.
4. **Evaluating Misclassification Costs:**
    - Becomes more complicated with more classes.
    - **Misclassification Types:**
        - For m classes, there are $m(m-1)$ types of misclassifications.
    - Constructing a matrix of misclassification costs can be prohibitively complex.
5. **Practical Implications:**
    - Complexity increases significantly with the number of classes.
    - Although the conceptual approach remains the same, the practical implementation for handling misclassification costs becomes challenging.

Judging Ranking Performance

- We now turn to the predictive goal of detecting, among a set of new records, the ones most likely to belong to a class of interest.
- Recall that this differs from the goal of predicting class membership for each new record.

Lift Charts for Binary Data

1. **Introduction to Lift Charts:**
   - Also known as lift curves, gains curves, or gains charts.
   - More commonly used for binary outcomes than for predicted continuous outcomes.
2. **Purpose of Lift Charts:**
   - Determine how effectively we can "skim the cream" by selecting a small number of records to get a large portion of the responders.
3. **Input Required:**
   - A validation dataset that has been scored with the propensity of each record to belong to a given class.
4. **Use Case for Rare and Important Classes:**
   - Suitable for situations where a particular class is rare and of much more interest than the other class, such as:
     - Tax cheats
     - Debt defaulters
     - Responders to a mailing
5. **Classification Model Goals:**
   - Sift through records and sort them by likelihood of belonging to the important class.
   - Make informed decisions based on sorted data.
6. **Practical Applications:**
   - **Example 1: Tax Cheats:**
     - Decide how many and which tax returns to examine based on sorted likelihoods of tax cheats.
     - Estimate the extent of encountering non-cheaters as we move through sorted data.
   - **Example 2: Mailing Campaign:**
     - Use sorted data to target potential customers within a limited budget for a mailing campaign.
7. **Rank Ordering Goal:**

- Obtain a rank ordering among records according to their class membership propensities.
- Helps prioritize actions based on likelihoods.

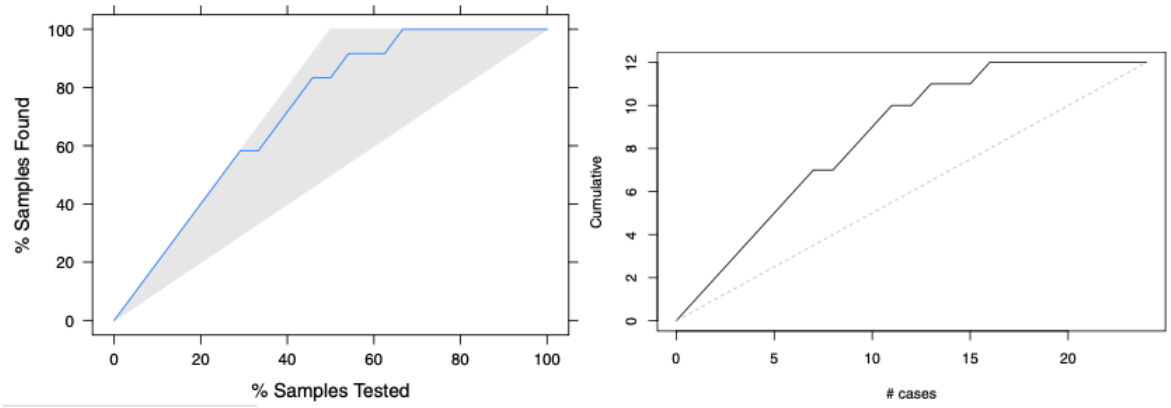
Decile Lift Charts

Sorting by Propensity:

1. Sort records by their propensity to belong to the important class (C_1), in descending order.
2. **Cumulative Computation:**
   - For each row, compute the cumulative number of C_1 members (Actual Class = C_1).
   - **Example:**
     - Table 5.6 shows 24 records ordered by descending class "1" propensity.
     - Right-most column accumulates the number of actual "1"s.
3. **Plotting the Lift Chart:**
   - Plot the cumulative number of C_1 members (y-axis) against the number of records (x-axis).
4. **Creating Lift Charts in R:**
   - **Caret Library:**
     - Straightforward and easy to use.
     - Cannot be used for numerical outcome variables.
   - **Gains Library:**
     - Another option for creating lift charts.
   - **Figure 5.6:**
     - Shows lift curves and R code for creating lift charts using both the caret and gains libraries.
     - Note: Caret package lift chart uses percentage on the y-axis.

Obs	Propensity of 1	Actual Class	Cumulative Actual Class
1	0.995976726	1	1
2	0.987533139	1	2
3	0.984456382	1	3
4	0.980439587	1	4
5	0.948110638	1	5
6	0.889297203	1	6
7	0.847631864	1	7
8	0.762806287	0	7
9	0.706991915	1	8
10	0.680754087	1	9
11	0.656343749	1	10
12	0.622419543	0	10
13	0.505506928	1	11
14	0.471340450	0	11
15	0.337117362	0	11
16	0.217967810	1	12
17	0.199240432	0	12
18	0.149482655	0	12
19	0.047962588	0	12
20	0.038341401	0	12
21	0.024850999	0	12
22	0.021806029	0	12
23	0.016129906	0	12
24	0.003559986	0	12

543



5. **R Code Example:**

   - **Using Caret Library:**

```
library(caret)
model <- train(Class ~ ., data = trainingData, method = "rpart")
predictions <- predict(model, validationData, type = "prob")
liftChart <- lift(ActualClass ~ propensities, data = predictions)
plot(liftChart)
```

   - **Using Gains Library:**

```
library(gains)
gainsCurve <- gains(validationData$ActualClass, propensities)
plot(gainsCurve)
```

Interpreting the lift chart

1. **Ideal Performance:**
   - All 1's ranked at the beginning with the highest propensities.
   - All 0's ranked at the end.
   - **Ideal Lift Chart:**
     - Diagonal line with slope 1 until all 1's are accumulated, then horizontal.
2. **Best Possible Classifier:**
   - Overlaps with the existing curve at the start.
   - Continues with a slope of 1 until reaching all the 1's.
   - Then continues horizontally to the right.
3. **Useless Model:**
   - Assigns propensities randomly.
   - **Random Assignment:**

  - Shuffles 1's and 0's randomly in the Actual Class column.
  - Increases cumulative number of 1's on average by $\frac{\#1\text{'s}}{n}$ per row.
- **Reference Line:**
  - Diagonal line from (0,0) to (24,12) in Figure 5.6.
  - Expected number of 1's if records are selected randomly.
  - Serves as a benchmark for evaluating model performance.

4. **Reading a Lift Chart:**
   - **For a Given Number of Records (x-axis):**
     - Lift curve value (y-axis) indicates performance compared to random assignment.
   - **Example:**
     - **Figure 5.6:**
       * Top 10 records using the model: About 9 correct (or 18% using caret package lift curve).
       * Random selection of 10 records: Expect 5 correct.
       * Model lift: $\frac{9}{5} = 1.8$.
5. **Interpreting Lift:**
   - High lift when acting on a few records indicates a good classifier.
   - Lift decreases as more records are included.

Decile lift charts

1. **Decile Chart Overview:**
   - Aggregates lift information into 10 buckets (deciles).
   - Widely used in direct marketing predictive modeling.
2. **Reading the Decile Chart:**
   - **Y-Axis:**
     - Shows the factor by which the model outperforms a random assignment of 0's and 1's.
   - **Example: Figure 5.7:**
     - Dots on the y-axis indicate performance for each decile.
3. **Interpreting Deciles:**
   - **Leftmost Bar:**
     - Taking 8% of the top-ranked records by the model (most probable 1's) yields twice as many 1's as a random selection of 8% of records.
   - **General Performance:**
     - The decile chart shows model performance compared to random selection for each decile.
   - **Top 29% Records:**

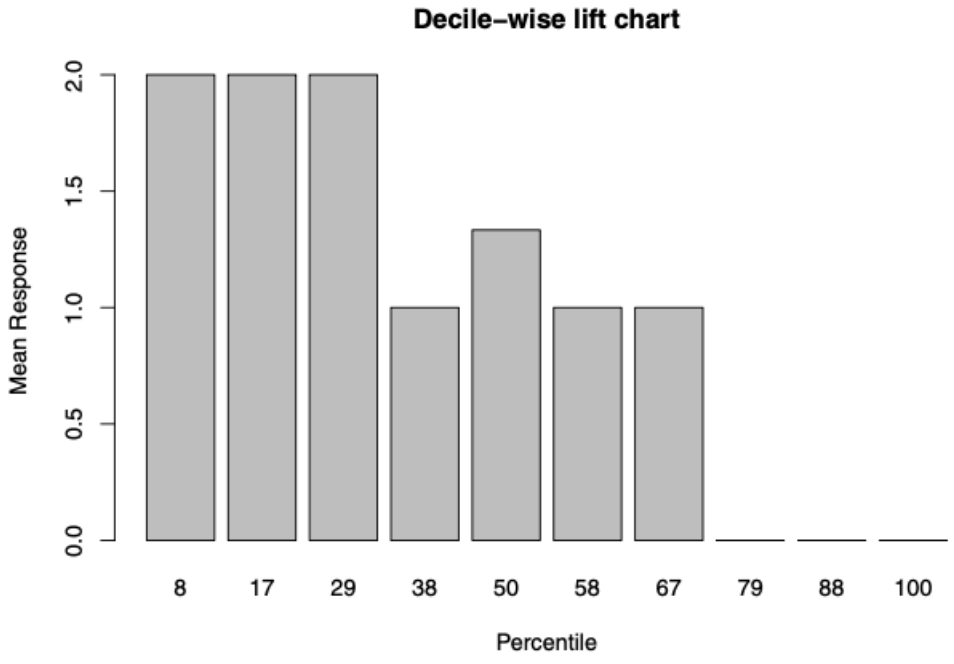
595 – The model can select up to the top 29% records with the highest propensities and still
596 perform twice as well as random.

597 4. **Using the Decile Chart:**

598 • Helps visualize and interpret the effectiveness of a model in ranking and selecting records.
599 • Provides a clear comparison of model performance versus random selection across deciles.

```
# first option with 'caret' library:
library(caret)
lift.example <- lift(relevel(as.factor(actual), ref="1") ~ prob, data = df)
xyplot(lift.example, plot = "gain")

# Second option with 'gains' library:
library(gains)
df <- read.csv("liftExample.csv")
gain <- gains(df$actual, df$prob, groups=dim(df)[1])
plot(c(0, gain$cume.pct.of.total*sum(df$actual)) ~ c(0, gain$cume.obs),
     xlab = "# cases", ylab = "Cumulative", type="l")
lines(c(0,sum(df$actual))~c(0,dim(df)[1]), col="gray", lty=2)
```

600



601

602 Beyond Two Classes

603 • A lift chart cannot be used with a multiclass classifier

604 – Unless a single “important class” is defined and the classifications are reduced to “important”
605 and “unimportant” classes.

Lift Charts Incorporating Costs and Benefits

Incorporating Benefits and Costs in Lift Charts

When the benefits and costs of correct and incorrect classification are known, they can be integrated into the lift chart to aid in decision-making.

Procedure:

1. **Sort Records:**
   - Arrange records in descending order of predicted probability of success (where success = belonging to the class of interest).
2. **Record Cost/Benefit:**
   - For each record, note the cost or benefit associated with the actual outcome.
3. **Calculate Initial Coordinates:**
   - For the highest propensity (first) record:
     - x-axis value: 1
     - y-axis value: Cost or benefit (computed in step 2)
4. **Compute Cumulative Cost/Benefit:**
   - For each subsequent record, calculate the cost or benefit associated with the actual outcome.
   - Add this cost or benefit to the cumulative total for the previous record.
   - This sum is the y-axis coordinate for the current record.
   - x-axis value: Position of the current record in the sorted list.
5. **Connect Points:**
   - Repeat the process for all records.
   - Connect all points to form the lift curve.
6. **Reference Line:**
   - A straight line from the origin to the point where $y =$ total net benefit and $x = n$ (where n is the number of records).

Example: Incorporating Costs and Benefits

- **Mailing Campaign Scenario:**
  - Cost of mailing to a person: \$0.65
  - Value of a responder: \$25
  - Overall response rate: 2%
  - List size: 10,000

Net Value Calculation: - Expected net value:

$$(0.02 \times \$25 \times 10,000) - (\$0.65 \times 10,000) = \$5,000 - \$6,500 = -\$1,500$$

- The y-value at the far right of the lift curve ($x = 10,000$) is -1,500. - The slope of the reference line from the origin will be negative.

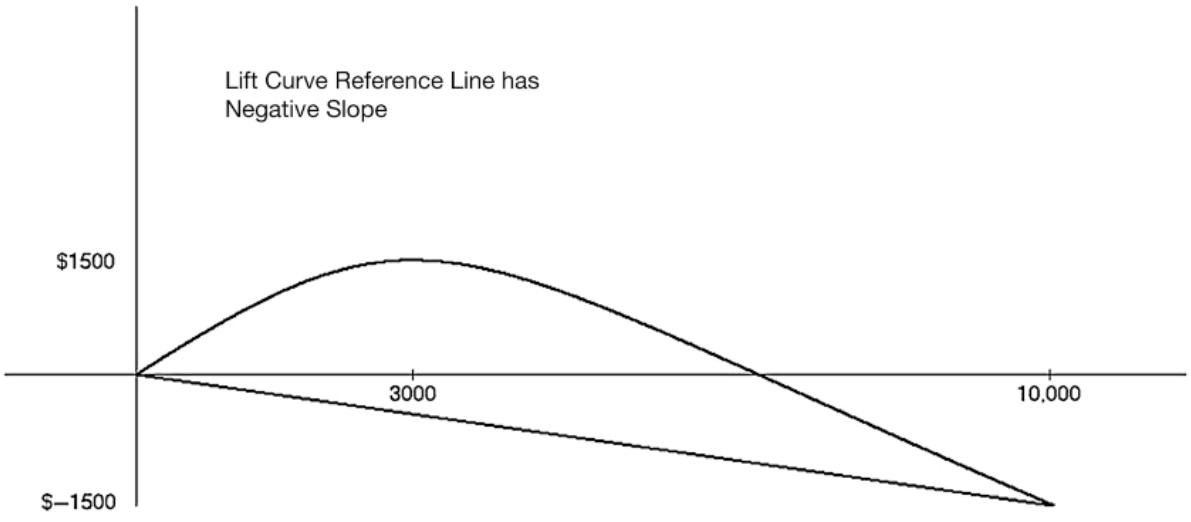
Interpreting the Lift Curve:

- Optimal point: Where the lift curve is at its maximum.
  - In **Figure 5.8**, optimal mailing would be to about 3,000 people.

Visual Representation:

- **Construct the Lift Curve:**
  - Follow steps to plot cumulative cost/benefit on the y-axis against the number of records on the x-axis.
- **Reference Line:**
  - Draw a line from the origin to the total net benefit at $x = n$. A negative slope indicates a negative overall net value.

This method provides a clear visual way to incorporate costs and benefits into decision-making and helps identify the optimal operating point for a given campaign or classification task.



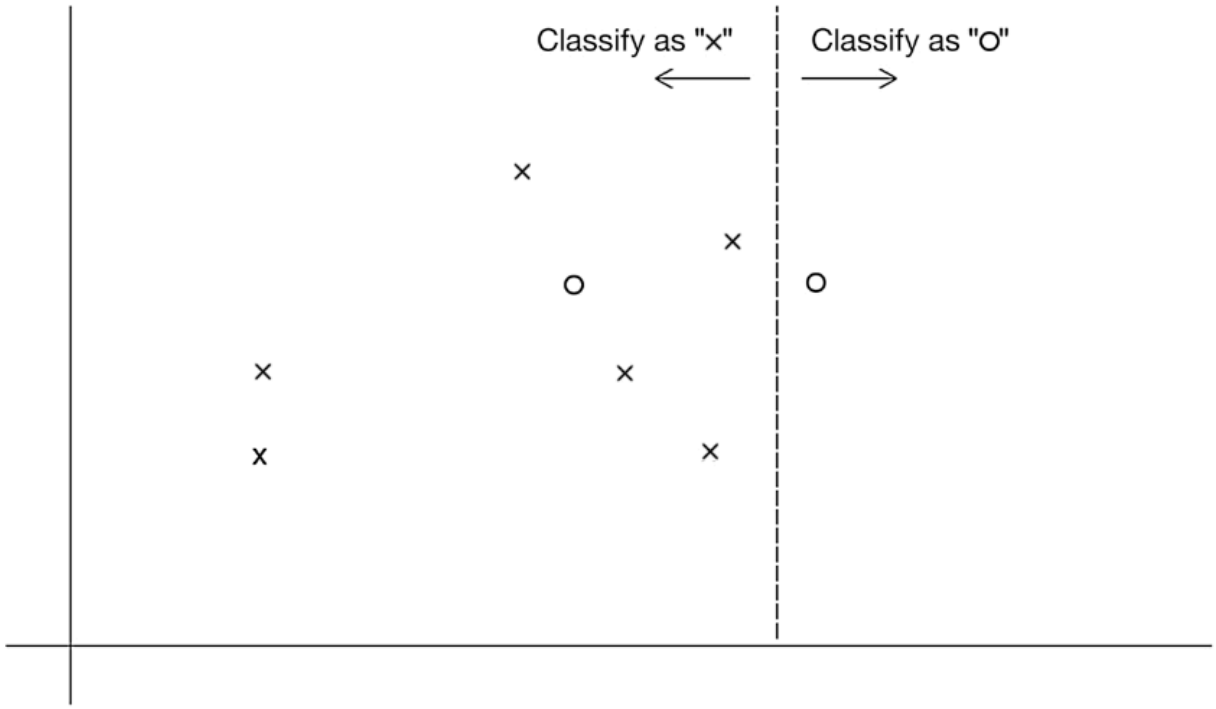
Lift as a Function of Cutoff

- We could also plot the lift as a function of the cutoff value.
- The only difference is the scale on the x-axis.
- When the goal is to select the top records based on a certain budget, the lift vs. number of records is preferable.
- In contrast, when the goal is to find a cutoff that distinguishes well between the two classes, the lift vs. cutoff value is more useful.

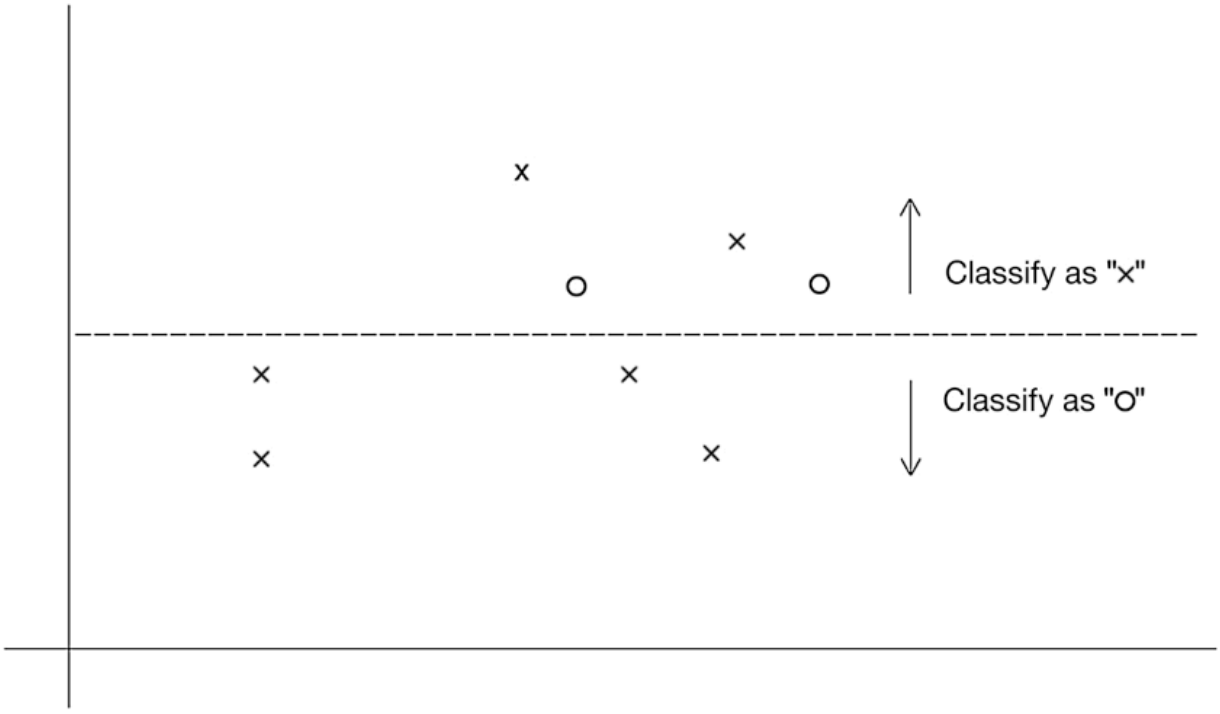
Oversampling

Stratified Sampling and Oversampling

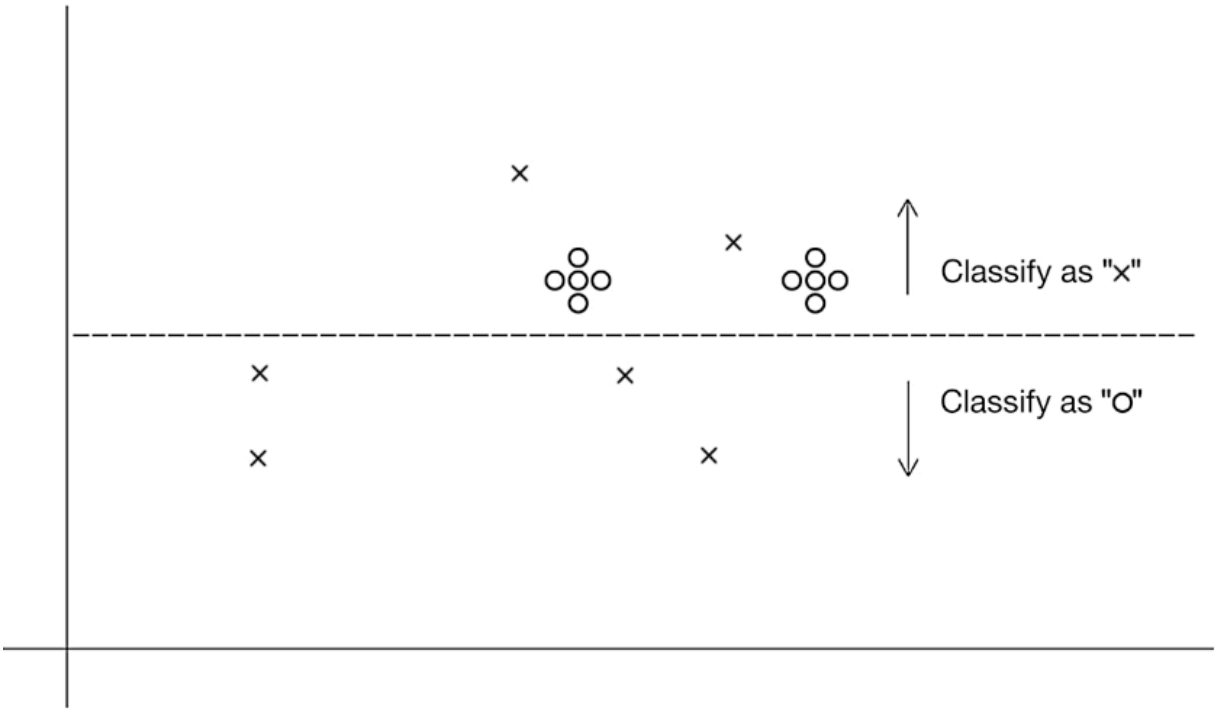
1. **Problem with Simple Random Sampling:**
   - When classes are present in very unequal proportions, simple random sampling may yield too few instances of the rare class.
   - This creates difficulty in distinguishing the rare class from the dominant class.
2. **Solution: Stratified Sampling (Oversampling):**
   - **Definition:**
     - A method used to oversample records from the rarer class to improve classifier performance.
   - **Also Known As:**
     - Weighted sampling.
     - Undersampling (since the more plentiful class is undersampled relative to the rare class).
   - **Focus:**
     - Highlight rare yet important events (e.g., responders to mailings, fraudsters, defaulters).
3. **Example:**
   - **Figure 5.9:**
     - Non-responders (×) and responders (◦).
     - Two predictors as axes.
     - Dashed vertical line for classification with equal costs results in one misclassification.



4. **Incorporating Misclassification Costs:**
   - **Equal Cost Assumption:**
     - Misclassification cost is low (vertical line).
   - **Unequal Cost Assumption:**
     - Failing to classify a responder (◦) as costly as five times failing to classify a non-responder (×).
     - Horizontal line for better classification with misclassification costs factored in.



5. **Oversampling Methods:**
   - **Figure 5.11:**
     – Automatically determines appropriate classification line by oversampling responders (◦).



- **Methods:**
  - Take a higher sample of the rare class (sampling with replacement if necessary).
  - Replicate the existing instances of the rare class.
  - Equal numbers of responders and non-responders can be a practical approach in cases of very low response rates.

6. **Adjusting for Oversampling:**
   - **Avoiding Oversampling Bias:**
     - Use one of the following two methods to assess and predict model performance:
       1. **Score Model on a Non-Oversampled Validation Set:**
          * Selected via simple random sampling.
       2. **Score Model on an Oversampled Validation Set:**
          * Reweight results to remove oversampling effects.
7. **Preferred Method:**
   - Method 1 (scoring against non-oversampled data) is more straightforward and easier to implement.

Implementation Steps:

1. **Oversampling:**

- Increase the proportion of records from the rare class.
- Example: If failing to classify a responder is five times more costly, oversample the responders accordingly.

2. **Evaluation:**
   - **Method 1:**
     - Use a validation set sampled randomly without oversampling.
     - Assess model performance on this set.
   - **Method 2:**
     - Use an oversampled validation set for scoring.
     - Adjust the results to account for oversampling (reweighting).

- By following these steps, classifiers can be trained more effectively with adequate representation of the rare class, and their performance can be accurately assessed.

Oversampling the Training Set

- How is weighted sampling done? When responders are sufficiently scarce that you will want to use all of them, one common procedure is as follows:
  1. First, the response and non-response data are separated into two distinct sets, or strata.
  2. Records are then randomly selected for the training set from each stratum. Typically, one might select half the (scarce) responders for the training set, then an equal number of non-responders.
  3. The remaining responders are put in the validation set.
  4. Non-responders are randomly selected for the validation set in sufficient numbers to maintain the original ratio of responders to non-responders.
  5. If a test set is required, it can be taken randomly from the validation set.

Evaluating Model Performance Using a Non-oversampled Validation Set

Training and Validating with Oversampled Data

Oversampling for Model Training: - **Purpose:** To ensure the rare class is adequately represented during training. - **Oversampling Methods:** - Increase sample size of the rare class (e.g., through replication or sampling with replacement).

Limitations for Model Evaluation: - **Bias in Evaluation:** - Oversampled data can exaggerate the number of responders, affecting the evaluation. - Leads to an unrealistic and biased estimate of model performance.

Unbiased Performance Estimation: - **Apply Model to Regular Data:** - Use a non-oversampled (regular) dataset to validate the model. - Ensures an unbiased and realistic estimate of performance.

Recommended Procedure: 1. **Train on Oversampled Data:** - Develop and train the classifier using the oversampled dataset. - Ensures the rare class is sufficiently represented and the model learns its characteristics.

2. **Validate on Regular Data:**
    - Validate the trained model using a regular, non-oversampled dataset.
    - Provides an unbiased estimate of predictive performance.

Practical Steps:

1. **Step 1: Train the Model**
    - Use oversampled data to train the model:

```
oversampled_data <- your_oversampled_dataset
model <- train(Class ~ ., data = oversampled_data, method = selected_method)
```

2. **Step 2: Validate the Model**
    - Apply the trained model to a regular validation dataset:

```
regular_validation_data <- your_validation_dataset
predictions <- predict(model, regular_validation_data)
confusionMatrix(predictions, regular_validation_data$Class)
```

3. **Step 3: Assess Performance**
    - Review the confusion matrix and other performance metrics generated from the validation:
        - Example: Accuracy, sensitivity, specificity.

Summary:

- **Training Phase:**
    - Utilize oversampled data to ensure the rare class is well-represented and the model learns effectively.
- **Validation Phase:**
    - Validate the model on regular data to get an unbiased estimate of how the model performs on actual, real-world distribution.

By following this approach, the model can leverage the benefits of oversampling during training while avoiding the pitfalls of biased performance estimates.

Evaluating Model Performance if Only Oversampled Validation Set Exists

I. Adjusting the Confusion Matrix for Oversampling

When dealing with oversampled data for both training and validation, it is important to adjust the confusion matrix to reflect the true proportions of the rare class in the original population.

Example Scenario: - Response rate in the original data: 2% - Response rate in oversampled data: 50% (25 times higher) - Oversampling weights: - Responders: 50 / 2 = 25 - Non-responders: 50 / 98 ≈ 0.5102

Oversampled Validation Confusion Matrix:

Actual	Actual 0	Actual 1	Total
Predicted 0	390	80	470
Predicted 1	110	420	530
Total	500	500	1000

Adjust the Counts: - Divide the number of responders by 25 - Divide the number of non-responders by 0.5102

Reweighted Confusion Matrix:

Actual	Actual 0	Actual 1	Total
Predicted 0	$\frac{390}{0.5102} = 764.4$	$\frac{80}{25} = 3.2$	767.6
Predicted 1	$\frac{110}{0.5102} = 215.6$	$\frac{420}{25} = 16.8$	232.4
Total	980	20	1000

Adjusted Performance: - Misclassification rate = $\frac{3.2+215.6}{1000} = 21.9\%$ - Records classified as 1s = $\frac{215.6+16.8}{1000} = 23.24\%$

II. Adjusting the Lift Curve for Oversampling

Steps to Adjust the Lift Curve: 1. **Sort Records:** - Sort validation records by predicted probability of success.

2. **Record Cost/Benefit:**
   - For each record, note the cost or benefit associated with the actual outcome.
3. **Adjust Value by Oversampling Rate:**
   - Divide the assigned cost/benefit by the oversampling rate (e.g., divide by 25 for responders).
4. **Calculate Initial Coordinates:**
   - For the highest probability record:
     - x-coordinate: 1
     - y-coordinate: Adjusted cost/benefit value after division in step 3.

5. **Compute Cumulative Adjusted Values:**
    - For each subsequent record:
        - Calculate the adjusted cost/benefit.
        - Add to the cumulative total of the previous record.
        - x-coordinate is the current record index.
        - The sum is the y-coordinate.
6. **Repeat Process:**
    - Continue until all records are accounted for.
7. **Draw Reference Line:**
    - Line from the origin to $y =$ total net benefit and $x = n$ (total number of records).

By following this procedure, the lift chart accurately reflects the performance of the model while accounting for the oversampling in the validation dataset.

Summary:

- **Oversampling in Training and Validation:**
    - Use oversampled data for training.
    - Adjust validation results to account for true class proportions.
- **Confusion Matrix Adjustment:**
    - Adjust counts in the confusion matrix using oversampling weights.
- **Lift Chart Adjustment:**
    - Sort records by predicted probability.
    - Adjust the cost/benefit by the oversampling rate.
    - Compute cumulative adjusted values and plot.

House Keeping

Announcement

Assignment

Assignment1: Assignment Title

- **What to do**
    - A
    - B

- **Format and Requirement**
  - PDF format, < ?? pages
  - file name should be include your student id and name
    - stuID_name_title.pdf (e.g. 1111111_ChungilChae_SelfIntroduction.pdf)
  - APA, Double-Space, No title page, 12 points, Reference section if necessary
  - Chinese name in English, English name, Student ID, Class name and section (e.g. GE2021, W09; MGS3001, W01)
- due date
  - by Date(DAY) 11:59PM
  - NO LATE SUBMISSION ALLOWED!!!!

Reference

